

Benchmarking Claude Code

Cost, skill effect and model selection — measured, priced, and where it surprised us

209 recorded repetitions · 149 in the cost grid (18 cells $n=5$, 2 at $n\approx 30$) · 4 models · internal LiteLLM

Last modified 2026-09-09 22:01 EDT

Contents

1 · SETUP AND METHOD

- 3 What a benchmark is made of
- 4 What is being measured
- 5 Terms in use
- 6 Benchmarking setup — architecture
- 7 Why this shape — rationale
- 8 How a task earns its place
- 9 What the verdict actually is

2 · PRICING

- 10 Model pricing — internal LiteLLM rate card

3 · FINDINGS

- 11 A prediction of ours that was falsified
- 12 Two factors drive token cost
- 13 Money reverses the token conclusion
- 14 Token-efficiency is not cost-efficiency
- 15 A subagent's calls land in the totals
- 16 Skill cost is a property of skill × model
- 17 Skill selection is reliable

4 · CONCLUSION

- 18 Model selection recommendation
- 19 Limitations, stated plainly

Every figure in this deck also appears in results/EVALUATION.md, which carries the full detail and the reproduction steps.

What a benchmark is made of

Six parts. Drop any one and you have a demo, a load generator, or a number nobody can defend.

TASKS

A defined unit of work, reproducible from a fixed definition into a fresh workspace.

`tasks/<id>/prompt.md + workspace/`

PROGRAMMATIC EVALUATOR

A verdict that is a command's exit code, not a model's opinion. This is what makes it a benchmark.

`hidden verdict/ → pytest -q`

CONTROLS

An arm that isolates the one variable under study, so an effect can be attributed to it.

`off / on / select arms`

REPETITION

The subject is non-deterministic, so one run is an anecdote. Report medians and spread.

`n per cell, CV reported`

ATTRIBUTION

Evidence of what ACTUALLY ran, so a result from something other than the thing tested is caught.

`stream-json transcript → confounds`

OBSERVABILITY

What it cost: tokens by cache tier, latency, the real model served. Measured, not estimated.

`Cortex supplies this`

Cortex is a COMPONENT of the harness — it supplies the observability data. It is not the benchmark, and it cannot score anything: it sees bytes on the wire, never whether the work was correct.

What is being measured

The subject under test is the Claude Code agent itself — not a model in isolation.

dimension	value
Agent under test	Claude Code CLI 2.1.257, headless (claude -p)
Task	one prompt + a fresh workspace; 1 repetition = 1 invocation
Verdict	a command's exit code — pytest -q. No LLM judge.
Measured on the wire	tokens + cache tiers via Cortex forward proxy
Skill measured	xlsx (2 discriminating tasks) + a no-skill control
Models priced	haiku-4-5, sonnet-4-6, sonnet-5, opus-5

No LLM judge is the load-bearing choice: without a programmatic verdict this is a load generator, not a benchmark.

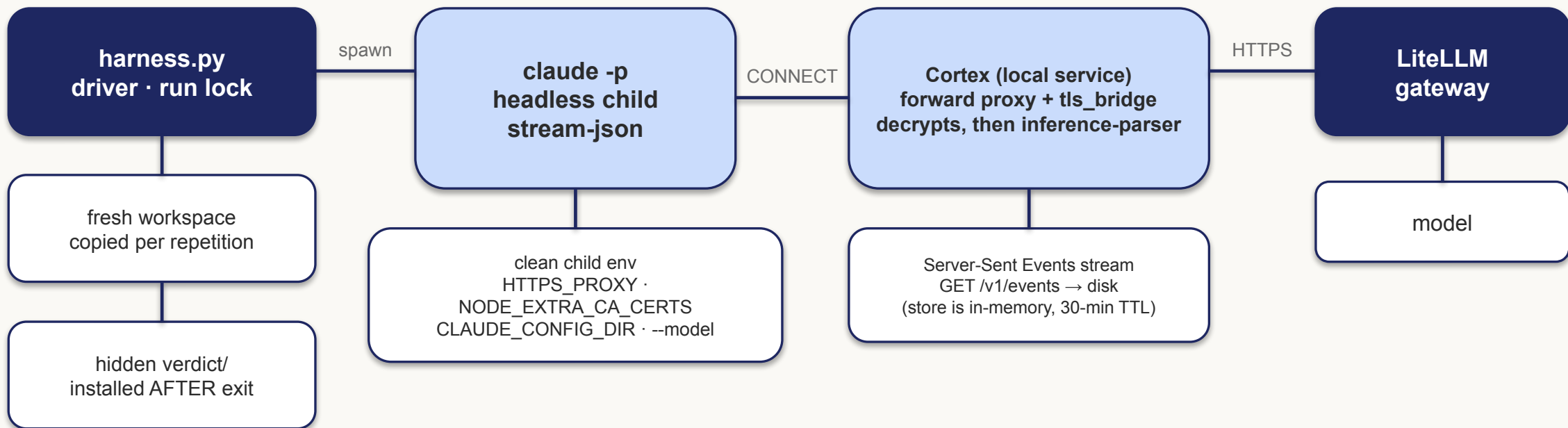
Terms in use

These were conflated early in the work; they are kept strictly separate.

term	meaning
Harness	the whole apparatus: tasks + driver + verdict + Cortex
Arm	off = skill unavailable (control) · on = skill invoked · select = none named
Cell	one (task × arm × model), measured over n repetitions — e.g. xlsx-fin-colors / ON / sonnet-5, n=5
Task / repetition	ONE headless `claude -p` invocation in a fresh workspace
LLM call	ONE /v1/chat/completions on the wire. A task makes SEVERAL — 2 to 29 here
Compliance task	ordinary request; hidden verdict checks a skill convention was followed
Selection task	names no skill; verdict is whether the model chose the right one
Confound	a repetition whose measurement is untrustworthy — reported, never averaged
Token-efficiency	tokens consumed per SOLVED task — what the context window and rate limits see
Cost-efficiency	dollars per SOLVED task — token-efficiency weighted by that model's unit price
Cost per solved task	median ÷ pass rate, so a model is charged for its failures

‘workload-harness’ (hyphenated) is a proper noun for an unrelated upstream project — never used here as a common noun.

Benchmarking setup



one NDJSON row per repetition · whitelisted fields only, never raw prompts

Three measurement sources, because none can answer another's question: the verdict says whether it worked, Cortex says what it cost, the transcript says which tools and skills actually ran.

Transport: HTTPS_PROXY is named "https" but its value is http://127.0.0.1:47600 — the hop to the local Cortex service is plaintext HTTP CONNECT on loopback. tls_bridge then terminates TLS with its own CA (hence NODE_EXTRA_CA_CERTS) so the request can be parsed, and Cortex makes the real HTTPS connection outbound to the gateway.

Why this shape

decision	rationale — each one was measured, not assumed
Per-invocation isolation	operator settings.json never touched; one Cortex sees ALL machine traffic, so global wiring would let interactive typing pollute a run
Forward proxy + tls_bridge	the reverse role records events but leaves plugins=[] — no tokens. Only forward+bridge dispatches the parser
CLAUDE_CONFIG_DIR, not HOME	HOME is not the skill lookup root; four curated-HOME arms reported an identical 60 slash commands
Pin --model, verify on wire	the model actually used came from an ambient env var disagreeing with settings.json
Exclusive run lock	Cortex correlates by TIME WINDOW on a shared proxy, so overlapping runs interleave — observed, then made impossible
Whitelisted artefacts	Cortex events carry full prompts and completions on an unauthenticated API

How a task earns its place

9 of the first 11 tasks written were discarded. A discarded task is a result.

RULE 1 — tamper-proof, not merely green

Pass requires exit 0 AND every test file
byte-identical to what shipped.

Deleting the test, or rewriting it to assert
the bug, also makes pytest green.

RULE 2 — pre-screen on the OFF arm

A task the agent already passes WITHOUT
the skill is not measuring the skill.

Hidden verdicts: a visible test is a spec —
the agent reads it and both arms pass.

The reusable heuristic

rule kind	example	outcome
Arbitrary / house-specific	xlsx: blue text = hardcoded input	DISCRIMINATES
Objectively better practice	docx: real bullets · pptx: size hierarchy	passes unaided
Path-dependent	docx rules correct footguns in the library the skill itself mandates	cannot discriminate

What the verdict actually is

Plain pytest, run by the repo's own interpreter. A pass needs exit 0 AND every test file byte-identical to what shipped — deleting the test, or rewriting it to assert the bug, also makes pytest green.

task	verdict file	visible to agent?	asserts
cortex-pyfix-001	workspace/test_billing.py	YES — tests ARE the spec	6 tests over 2 seeded bugs: a percentage treated as a fraction, and a dropped remainder
xlsx-fin-colors	verdict/test_compliance.py	no — installed after exit	2 tests: hardcoded inputs are blue-font; formula cells are not blue
xlsx-fin-font-clean	verdict/test_compliance.py	no — installed after exit	3 tests: one consistent professional font; no #REF!-class literals; saving and ratio are formulas

The visible / hidden split is the load-bearing choice. cortex-pyfix SHOULD show its tests, because satisfying them is the task. A COMPLIANCE verdict must stay hidden — it enumerates the conventions being checked, so an agent that can read it simply complies and BOTH arms pass, measuring nothing.

Verdicts are also tamper-checked before the hidden file is installed, so the hidden test cannot itself be counted as a modification.

Model pricing — internal LiteLLM

benchmarked alias	gateway entry	input \$/1M	output \$/1M	vs sonnet-5
claude-haiku-4-5-20251001	aws/claude-haiku-4-5	0.76	3.80	0.50×
claude-sonnet-4-6	aws/claude-sonnet-4-6	2.28	11.40	1.50×
claude-sonnet-5	aws/claude-sonnet-5	1.52	7.60	1.00×
claude-opus-5	aws/claude-opus-5	3.80	19.00	2.50×

Transcribed by hand from the gateway model pages on 2026-09-09. That page needs interactive internal web authorization and fills itself by script, so no automated pull is possible or attempted.

sonnet-5 is priced at 2/3 of sonnet-4-6 — the single most consequential fact in the analysis.

The cache caveat

The gateway publishes only Input and Output rates, but 81–97% of our prompt tokens are CACHE READS.

- Scenario A — no discount: every prompt token at the Input rate (upper bound)
 - Scenario B — standard Anthropic/Bedrock convention: cacheRead ×0.10, cacheWrite ×1.25
- B lands ~4–5× below A. Which the gateway actually bills is UNVERIFIED.

The model ranking is identical under both, so the recommendation does not depend on resolving it.

A prediction of ours that was falsified

We argued a compliance task cannot rank models: the OFF arm is pinned at 0 by the pre-screen and the ON arm at 100 because the skill states the answer. That held only for the two mid-tier models we had tested.

model	xlsx-fin-colors OFF → ON	xlsx-fin-font-clean OFF → ON
haiku-4-5	0.00 → 1.00	0.00 → 0.52 (n=29) ← fails even when TOLD
sonnet-4-6	0.00 → 1.00	0.00 → 1.00
sonnet-5	0.00 → 1.00	0.00 → 1.00
opus-5	0.37 (n=30) → 1.00 ← knows it UNAIDED	0.00 → 1.00

Every cell is n=5 except the two annotated ones — the only two rates here that are neither 0 nor 1, so the only two sampling error could overturn. They were taken to n≈30: both ROSE (0.40→0.52, 0.20→0.37) and both 95% intervals roughly halved. The n=5 readings were pessimistic, not misdirected.

“The measure is saturated” is a claim about the models you happened to test — not a claim about the task.

Two factors drive token cost

$$\text{tokens per task} = \text{tokens per LLM CALL (a MODEL property)} \times \text{LLM calls per task (a TASK property)}$$

Factor 1 — tokens per LLM call: ratio vs sonnet-4-6, across five structurally different cells.
“spread” = highest ratio minus lowest. Small spread means the ratio is a property of the MODEL, not of the task (≤0.15 is reported as CONSTANT).

model	range across the 5 cells	spread	reading
haiku-4-5	0.95 – 1.04	0.09	≈ same context per LLM call
sonnet-5	1.18 – 1.24	0.05	≈1.21× more per LLM call
opus-5	0.88 – 0.93	0.05	≈0.90× — leaner per LLM call

Factor 2 — LLM calls per task: NOT constant. 0.40–1.00× for haiku, 0.83–2.00× for opus-5, by task.

One TASK is one `claude -p` run making SEVERAL LLM calls (2–29), each re-sending the growing conversation.
A raw token total multiplies the two factors and hides both.

That re-sending is also why cache reads are 81–97% of prompt tokens in every cell.

Money reverses the token conclusion

\$ per SOLVED task (scenario B). Cost per solved = median tokens ÷ pass rate, so failures are charged.

task	haiku-4-5	sonnet-4-6	sonnet-5	opus-5
xlsx-fin-colors (ON)	0.0386	0.1746	0.1608	0.2719
xlsx-fin-font-clean (ON)	0.0822	0.1896	0.1261	0.2486
cortex-pyfix-001 (no skill)	0.0313	0.0839	0.0588	0.1030

haiku's 0.0822 already charges it for a 0.52 pass rate (n=29) — cheapest even after paying for its failures.

opus-5 used the FEWEST tokens on two of three cells — and is the MOST EXPENSIVE on all three.

sonnet-5 uses MORE tokens than sonnet-4-6 yet costs LESS on 2 of 3 cells, at 2/3 the unit price — but only ONE of the three separates them (canary, p=0.008). Colors is a TIE (p=0.78); font-clean flipped sign on a re-run. Don't quote 2/3.

Same cells under scenario A (no cache discount). haiku stays cheapest and opus-5 dearest everywhere — the two sonnets appear to SWAP on.xlsx-fin-colors, but neither ordering is established there (p = 0.78 under B, 0.40 under A):

task	haiku-4-5	sonnet-4-6	sonnet-5	opus-5
xlsx-fin-colors	0.1409	0.6677	0.8920	1.1625
xlsx-fin-font-clean	0.3501	0.8484	0.6625	1.1259
cortex-pyfix-001	0.1597	0.4582	0.3113	0.5649

Token-efficiency is not cost-efficiency

	token-efficiency	cost-efficiency
measures	tokens per SOLVED task	dollars per SOLVED task
binding when	context window, rate limits, latency	you are paying the bill
driven by	how much context and how many calls	the same, weighted by unit price

They diverge because unit price spans 5× (haiku \$0.76 → opus-5 \$3.80 per 1M input), which swamps the ~2× spread in token counts. On cortex-pyfix-001 the two rankings are EXACTLY INVERTED:

model	tokens/solved task	rank	\$/solved task	rank
opus-5	144,834	1st	0.1030	4th
sonnet-4-6	196,634	2nd	0.0839	3rd
sonnet-5	200,732	3rd	0.0588	2nd
haiku-4-5	204,034	4th	0.0313	1st

opus-5 is the MOST token-efficient and the LEAST cost-efficient model tested. Quoting one number and calling it “efficiency” picks the answer by accident.

A subagent's calls land in the totals — and can't be separated out

The child's HTTPS_PROXY is inherited by its subprocesses, so a subagent's traffic crosses the same proxy.

measure	effect of downstream work
total tokens / dollars	CORRECT — the calls really happened and are really billed
tokens per LLM call	unaffected — still that model's per-call average
LLM calls per task	INFLATED — the “task” is no longer a single agent loop
any skill-overhead ratio	INVALID — it compares two different amounts of work

Magnitude where it occurred — affected repetitions vs others in the same cell:

cell	with downstream	without
pptx-body-left-aligned ON	26 calls / 1,255,632 tok	10 calls / 458,514 tok
pptx-size-contrast ON	36 calls / 1,908,134 tok	10 calls / 491,474 tok
select-deck SELECT	46 calls / 2,485,834 tok	7 calls / 302,900 tok

2.7×–8.2×. The detector that should have flagged these matched a tool named `Task`, but this build names it `Agent` — so it never fired and 5 repetitions passed as clean. Fixed and unit-tested. The xlsx COST PROFILE is unaffected: none of the 5 fall in its 20 cells.

Skill cost is a property of skill × model

skill-on ÷ skill-off, same skill (xlsx), same task, same model. Tokens AND dollars — they differ, because the input/output/cache mix shifts between arms even at a fixed unit price.

task / measure	haiku-4-5	sonnet-4-6	sonnet-5	opus-5
xlsx-fin-colors — tokens	2.88×	1.54×	1.98×	0.94×
xlsx-fin-colors — dollars	1.97×	1.44×	1.80×	1.04×
xlsx-fin-font-clean — tokens	3.33×	2.06×	1.44×	1.01×
xlsx-fin-font-clean — dollars	2.21×	1.69×	1.32×	1.08×

“What does this skill cost?” has no single answer.

On opus-5 the skill adds only 6–8% in dollars; on haiku it roughly doubles the bill. Opus-5 barely notices it because it already works ~2× the calls WITHOUT the skill (10 vs 8 on colors), so the guidance replaces exploration rather than adding to it. That is why its token ratio can dip below 1.0.

Caveat: this rests on ONE skill. Whether the effect belongs to opus-5 or to xlsx is currently indistinguishable — three more tasks on docx / pptx / pdf would separate them.

Skill selection is reliable

All four candidate skills present, prompt names none, verdict read from the transcript.
sonnet-4-6, n=3 per task — 12/12 correct, 0 confounded.

task	cue strength	expected	fired
select-spreadsheet	names the artifact	xlsx	xlsx ×3
select-deck	names the artifact	pptx	pptx ×3
select-implicit-sheet	never says “spreadsheet”	xlsx	xlsx ×3
select-none	plain bugfix	nothing	nothing ×3

A negative case is mandatory: measuring only true positives rewards a model that fires a skill on everything. Consequence — these four are now too easy to discriminate; extending needs ambiguous prompts.

Only here is the transcript authoritative: a MODEL-SELECTED skill is a real Skill tool call, whereas an explicit /skill-name is expanded client-side and never appears in the transcript at all.

Model selection recommendation

on the benchmark evidence, for skill-driven document work

ADOPT AS DEFAULT

claude-sonnet-5

100% pass on every task, and never ESTABLISHED as dearer than sonnet-4-6 (scenario B): decisively cheaper on the canary (30%, $p=0.008$); colors a TIE (8%, $p=0.78$); font-clean UNRESOLVED — it flipped sign on a re-run. Despite MORE tokens: 2/3 unit price absorbs 1.21×/call.

ONLY WITH A VALIDATOR

claude-haiku-4-5

Most cost-efficient everywhere, by 1.5–4.2× over the next cheapest, and fastest. But it failed 14 of 29 WITH the skill supplied, and 7 of those 15 assertion failures are invisible to review — incl. 3 that hardcoded values instead of formulas. Retry needs a validator.

NOT INDICATED HERE

claude-opus-5

Most expensive in all three cells (1.7–2.0× sonnet-5, scenario B), no pass-rate advantage. Genuinely the most token-efficient and the only model to solve a task unaided — may earn its premium on harder work.

Confidence: pass rates, the cheapest/dearest ordering and the token/cost inversion all hold across BOTH pricing scenarios; tokens/call holds across five independent cells. Absolute dollar figures are indicative — $n=5$ in 18 cells, cache billing unverified, and only 1 of the 3 sonnet-vs-sonnet cells separates the two models at all.

Limitations, stated plainly

limitation	consequence
One skill (xlsx)	every skill-specific conclusion is xlsx-specific; “free on opus-5” may be an xlsx property
Uneven n: 18 cells at 5, 2 at ≈30	chosen, not accidental — the 2 non-saturated pass rates got the reps. CV to 0.85, so a single cell's median is indicative; tokens/call constants are solid (5 cells agree)
Not every \$ gap is a finding	the sonnet-5 vs sonnet-4-6 saving is established on only 1 of 3 cells (canary, p=0.008); colors is a tie (p=0.78, ~617 reps/cell to settle) and font-clean is unresolved
One cell is NOT stationary	re-run 23 h later, sonnet-5 took ×3.4 cost / ×4.1 tokens on font-clean for the same output while sonnet-4-6 held to 3%. Gateway change vs fat tail: indistinguishable — the wire only echoes the alias
Cache billing unverified	4–5× on ABSOLUTE cost; no headline ranking changes, and the sonnets' apparent swap on colors is an unresolved comparison, not a real reversal
Two tasks, one narrow genre	financial-spreadsheet formatting. Not a general coding benchmark
aws/ vs bare alias pricing	assumed identical; unprovable with a non-admin key
Wall-clock not load-controlled	gateway-dependent; treat as indicative only

If the gateway does NOT discount cache reads, absolute costs rise ~4–5× and cache-heavy agentic use becomes far more expensive in aggregate — the ranking survives, but budget planning changes materially. If harder tasks were added, haiku’s reliability gap would likely widen and opus-5 might start earning its premium.

Full detail, every figure and the reproduction steps: [results/EVALUATION.md](#)